

Smart Manufacturing IoT-Based Predictive Maintenance and Anomaly Detection

Group 22

Aomsin Nimsombun, Colin Xiong, Kaoklong Panitpongpun, Daniel Benjamin
MANU 465 – University of British Columbia

1 Introduction

1.1 Problem Statement and Motivation

Modern smart manufacturing systems rely on IoT sensors to continuously monitor equipment conditions. Despite rich sensor data—including temperature, vibration, pressure, and humidity—many facilities depend on reactive or scheduled maintenance strategies that result in unexpected breakdowns, unnecessary servicing, and significant operational costs. Studies estimate that unplanned downtime costs industrial manufacturers hundreds of thousands of dollars per hour, making even marginal improvements in early failure detection highly valuable [1].

This project develops and evaluates machine learning models for predicting whether maintenance is required based on IoT sensor data. Additionally, unsupervised anomaly detection is explored to identify abnormal machine behavior prior to failure events, addressing scenarios where labeled failure data may be unavailable. The goal is to demonstrate how data-driven models can support the transition from reactive to proactive maintenance scheduling in smart manufacturing.

1.2 Project Scope

This project applies five ML techniques of increasing complexity: (1) **Logistic Regression** as an interpretable baseline; (2) **Random Forest** for non-linear feature interactions; (3) **ANN** with TensorFlow/Keras for complex pattern recognition; (4) **PCA** for dimensionality reduction; and (5) **Isolation Forest** for unsupervised anomaly detection. All models are evaluated with emphasis on precision-recall trade-offs relevant to maintenance, where missed failures carry higher cost than false alarms.

2 Dataset Description

This project utilizes the *Smart Manufacturing IoT-Cloud Monitoring Dataset* from Kaggle, simulating real-time sensor measurements from 50 industrial machines.

Table 1: Dataset overview.

Property	Description
Number of records	100,000
Number of machines	50 unique machines
Time resolution	1-minute intervals
Time range	69 days (Jan–Mar 2025)
Primary sensors	Temperature, Vibration, Pressure, Humidity
Additional features	Energy Consumption
Excluded (leakage)	Machine Status, Anomaly Flag, Downtime Risk, etc.
Target variable	<code>maintenance_required</code> (binary: 0/1)
Missing values	None

The dataset contains 13 columns in total. Non-predictive identifiers (`timestamp`, `machine_id`) and derived features that encode the target variable were removed during preprocessing (see Section 3.1).

2.1 Exploratory Data Analysis

The class distribution is imbalanced: 80.3% no-maintenance vs. 19.7% maintenance-required. As shown in Figure 1, `temperature` has the strongest individual correlation with the target ($r = 0.283$), followed by `vibration` ($r = 0.113$). Weak individual correlations for other features suggest non-linear combinations are important—confirmed by the superior performance of non-linear models. No missing values were found.

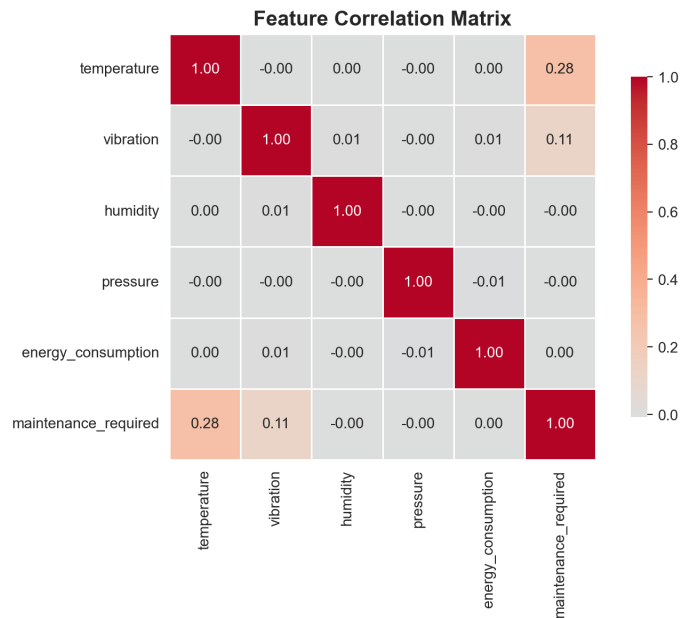


Figure 1: Correlation matrix of sensor features and maintenance target.

3 Methodology

3.1 Data Preprocessing

Preprocessing involved four steps: (1) dropping non-predictive identifiers (`timestamp`, `machine_id`) and derived features that encode the target variable; (2) retaining only 5 raw sensor features—temperature, vibration, humidity, pressure, and energy consumption; (3) stratified 80/20 train-test split (80,000/20,000 samples) preserving class proportions; and (4) standardizing features ($z = \frac{x-\mu}{\sigma}$) using `StandardScaler` fitted *only* on training data to prevent information leakage. Five derived features were excluded (`machine_status`, `anomaly_flag`, `predicted_remaining_life`, `downtime_risk`, `failure_type`) because they encode the target: e.g., `machine_status=2` directly indicates failure. Class imbalance (80.3% vs. 19.7%) was addressed via `class_weight='balanced'`.

3.2 Machine Learning Models

3.2.1 Model 1: Logistic Regression (Baseline)

Logistic Regression was chosen as the baseline for its interpretability and well-understood linear decision boundary. It models $P(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b)$, providing a clear reference point against which more complex models are compared. The L-BFGS solver was used with balanced class weights (giving $\sim 4\times$ more weight to the minority class) and max 1000 iterations. Five-fold cross-validation with F1-score—rather than accuracy, which is misleading for imbalanced data—assessed generalization stability.

3.2.2 Model 2: Random Forest with Hyperparameter Tuning

Random Forest was selected to capture non-linear feature interactions that a linear model cannot represent. It aggregates predictions from many decision trees, each trained on a bootstrap sample with random feature subsets, reducing overfitting through bagging [5]. `GridSearchCV` (3-fold CV) searched over `n_estimators` $\in \{100, 200\}$, `max_depth` $\in \{10, 20, \text{None}\}$, `min_samples_split` $\in \{2, 5\}$, selecting the best configuration by F1-score. Feature importance was extracted via Gini impurity to identify which sensors most strongly predict maintenance needs.

3.2.3 Model 3: Artificial Neural Network (TensorFlow/Keras)

A feed-forward ANN was built using TensorFlow/Keras [3] to test whether deeper non-linear modeling improves on the ensemble. The funnel architecture progressively compresses representations: input (5) $\rightarrow$ Dense(64, ReLU) $\rightarrow$ Dense(32, ReLU) $\rightarrow$ Dense(16, ReLU) $\rightarrow$ Dense(1, sigmoid). Training used Adam (lr=0.001), binary crossentropy, batch size 256, 20% validation split, balanced class weights, and Early Stopping (patience=10) monitoring validation loss with best-weight restoration—preventing overfitting by halting when validation performance stops improving.

3.2.4 Dimensionality Reduction: PCA

PCA was applied to evaluate the intrinsic dimensionality of the feature space by projecting data onto directions of maximum variance. A scree plot determined the components needed for 95% variance retention. To quantify the impact of dimensionality reduction, LR trained on PCA-reduced features was compared to the full-feature baseline.

3.2.5 Anomaly Detection: Isolation Forest

Isolation Forest [6] is an unsupervised algorithm that isolates anomalies via random partitioning—points requiring fewer splits are anomalies. It addresses the practical scenario where labeled failure data is unavailable. Contamination was set to 0.2 ($\approx$ observed maintenance rate), and performance was evaluated against true labels for comparison with supervised methods.

3.3 Evaluation Metrics

All models were evaluated on the held-out test set using accuracy, precision, recall, F1-score ($F_1 = 2 \cdot \frac{P \times R}{P + R}$), ROC-AUC, and confusion matrices. In predictive maintenance, recall is particularly important because missed failures can lead to unplanned downtime with higher cost than false alarms.

4 Results

4.1 Model Performance Summary

Table 2 presents the test set performance of all models. Exact metric values are reproduced from the accompanying Jupyter notebook (`code.ipynb`).

Table 2: Model performance comparison on the test set.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Naive Baseline	~ 0.80	—	—	—	—
Logistic Regression	0.6352	0.2974	0.6255	0.4032	0.6929
Random Forest (tuned)	0.8913	0.9989	0.4488	0.6194	0.7230
Neural Network (ANN)	0.8831	0.9029	0.4557	0.6057	0.7250
LR + PCA (5 PCs)	0.6352	0.2974	0.6255	0.4032	0.6929
Isolation Forest	0.7476	0.3644	0.3783	0.3712	0.6571

4.2 Logistic Regression

With balanced class weights, LR trades majority-class accuracy for minority-class recall: accuracy (0.6352) falls below the naive baseline (~ 0.80) as the model aggressively flags potential failures. It catches 0.6255 of true maintenance cases but at low precision (0.2974), missing 37% of 3,939 positive cases. Cross-validation F1 (0.4091 ± 0.0060) confirmed stable generalization. The linear

decision boundary limits performance: with the strongest correlation at only $r = 0.283$, a linear separator cannot capture the non-linear sensor interactions that drive maintenance needs.

4.3 Random Forest

After GridSearchCV (best: `n_estimators=100`, `max_depth=None`, `min_samples_split=2`), RF achieved F1 of 0.6194 and AUC of 0.7230—a substantial improvement over LR (0.4032). RF’s strategy differs markedly: near-perfect precision (0.9989) means virtually no false alarms, but recall of only 0.4488 means it misses over half of true cases, flagging failures only when very confident. Feature importance (Figure 2) identified temperature (0.365), vibration (0.224), and humidity (0.148) as the top three predictors ($\sim 74\%$ of total importance), with temperature and vibration together accounting for over 58% of predictive power.

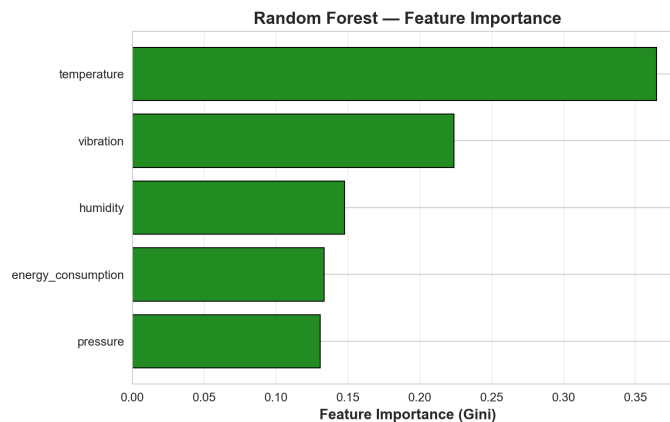


Figure 2: Random Forest feature importance ranking by Gini impurity.

4.4 Neural Network (ANN)

The ANN achieved F1 of 0.6057 and AUC of 0.7250—remarkably similar to RF (0.6194, 0.7230), suggesting both architectures have found a similar decision boundary given five sensor features. The ANN generates 193 false alarms while missing 2144 maintenance cases. Figure 3 shows smooth convergence with validation accuracy 0.8808 and loss 0.4705. Early Stopping halted at epoch ~ 90 (of 100 max); the small train-validation gap confirms healthy generalization.

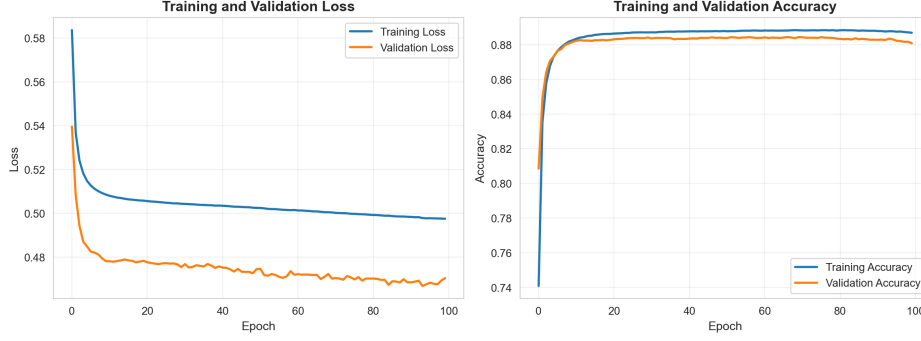


Figure 3: ANN training and validation curves showing convergence behavior.

4.5 PCA Analysis

All 5 of 5 components are needed to retain 95% of variance. PC1 explains 20.2% and PC2 20.1%—nearly equal—indicating the sensors are nearly orthogonal with minimal redundancy. LR on PCA-reduced features achieved identical performance ($\Delta F1 = 0.0000$) since all dimensions were retained, confirming no sensor can be dropped without losing information.

4.6 Isolation Forest

The unsupervised Isolation Forest achieved F1 of 0.3712 and AUC of 0.6571, with 2599 false alarms and 2449 missed failures. The F1 gap from IF (0.3712) to RF (0.6194) quantifies the value of labeled data. Despite this, AUC of 0.6571—well above random (0.5)—confirms that anomaly scoring captures meaningful structure without labels, providing a viable baseline when labeled failure data is scarce.

Figure 4 compares ROC curves for all models, with all supervised models well above the random baseline.

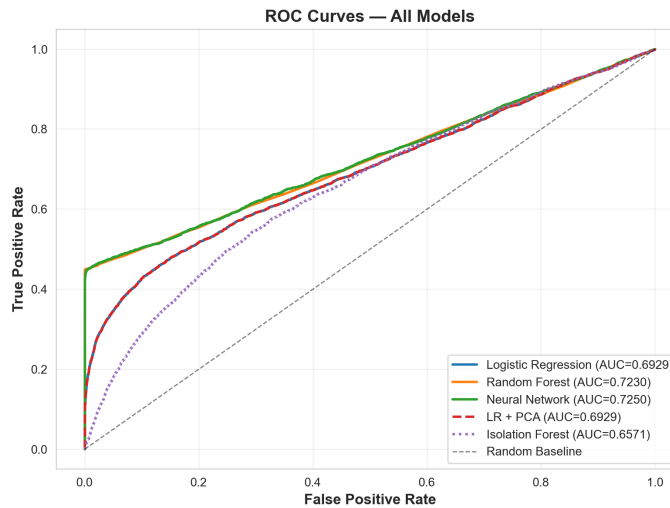


Figure 4: ROC curves for all models compared on the test set.

5 Discussion

5.1 Comparative Analysis

Three key patterns emerge. First, non-linear models (RF, ANN) substantially outperform LR, with F1 improving from 0.4032 to 0.6194—confirming that maintenance prediction depends on complex sensor interactions. Second, RF and ANN converge to nearly identical AUC (0.7230 vs. 0.7250) despite different architectures, suggesting a performance ceiling with five raw features. Third, the supervised–unsupervised gap (RF 0.6194 vs. IF 0.3712) quantifies the value of labeled data, while IF’s AUC (0.6571) confirms anomaly scoring captures real structure without supervision.

The models exhibit divergent precision-recall strategies: LR favors recall (0.6255) over precision (0.2974); RF near-perfect precision (0.9989) but lower recall (0.4488); ANN occupies the middle ground (0.9029, 0.4557). The optimal choice depends on the relative cost of unplanned downtime vs. unnecessary inspections.

5.2 Strengths

- **Comprehensive methodology:** Five techniques spanning linear models, ensembles, deep learning, dimensionality reduction, and unsupervised anomaly detection.
- **Rigorous evaluation:** Stratified split, cross-validation, multiple metrics (accuracy, precision, recall, F1, ROC-AUC), and a naive baseline.
- **Data leakage prevention:** Scaler fitted only on training data; derived features excluded.
- **Systematic tuning:** GridSearchCV for RF and Early Stopping for ANN prevent overfitting.
- **Actionable insights:** Feature importance from raw sensor measurements available in real-time.

5.3 Limitations

- **Simulated data:** The dataset is synthetic and may not capture real-world noise, sensor drift, or missing readings.
- **Limited feature set:** Only five raw sensor features were retained after removing derived features to avoid data leakage; richer feature engineering (e.g., rolling statistics, per-machine baselines) could improve performance.
- **No temporal modeling:** All models treat each record independently, ignoring sequential degradation patterns.
- **Single dataset:** Generalizability to different manufacturing environments remains unvalidated.

5.4 Potential Improvements

- **Temporal modeling:** LSTM or GRU networks could exploit sequential degradation trends.
- **Feature engineering:** Rolling window statistics, per-machine baselines, and interaction features could enrich the raw sensor feature set and potentially break through the ~ 0.62 F1 ceiling observed with raw features alone.

- **Explainability:** SHAP values could provide instance-level feature attribution.

6 Conclusion

This project applied five ML techniques to a 100,000-record IoT dataset using five raw sensor features. RF achieved the best F1 (0.6194) with near-perfect precision (0.9989); ANN reached comparable AUC (0.7250); LR (F1 = 0.4032) established a clear baseline; PCA confirmed no sensor redundancy; and Isolation Forest (AUC = 0.6571) demonstrated viable label-free monitoring. Feature importance identified temperature (0.365) and vibration (0.224) as the top sensors. By excluding derived features, these results represent realistic sensor-only performance. Future work should explore temporal modeling, richer feature engineering, and real-world validation.

References

- [1] R. K. Mobley, *An Introduction to Predictive Maintenance*, 2nd ed. Butterworth-Heinemann, 2002.
- [2] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [3] M. Abadi et al., “TensorFlow: A system for large-scale machine learning,” in *Proc. OSDI*, 2016, pp. 265–283.
- [4] “Smart Manufacturing IoT-Cloud Monitoring Dataset,” Kaggle. [Online]. Available: <https://www.kaggle.com/datasets>
- [5] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [6] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *Proc. IEEE Int. Conf. Data Mining*, 2008, pp. 413–422.